Multi Party Computation based on Boolean Secret Sharing

Elisabeth Oswald

ROADMAP

MPC
- Definitions
- Application as Side Channel Countermeasure

Secret Sharing for Probing Security
- Addition
- Special Multiplications
- General Multiplication
- ISW: “probing-secure” multiplication for (d,d) Boolean secret sharing

Multi Party Computation

MPC appears already during the advent of modern cryptography (1970s) as an idea: multiple, distrusting parties P_i wish to jointly compute a function $F()$.

Each party supplies some input(s), so P_i supplies X_i, to this function. The X_i may represent a single value, or a vector in practice.

Because the parties do not trust each other, they want to compute $F(X_1, \dots)$, without revealing their X_i: **Secure** MPC.

This looks impossible at first, but secret sharing is one way by which this can be achieved.

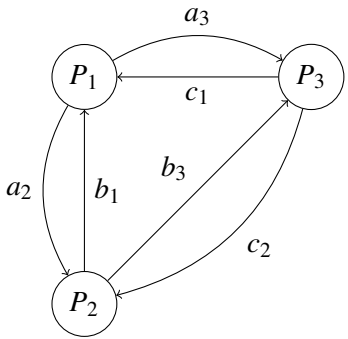
MPC, CONT.

Consider the computation of an addition, between three parties: P_1 with shares a_1, a_2, a_3 for their secret input a; P_2 with shares b_1, b_2, b_3 for their secret input b, and P_3 with shares c_1, c_2, c_3 for their secret input c. They could distribute shares such that each party P_i ends up having the $i-th$ share of each other party (e.g. P_1 has a_1, b_1, c_1, see picture below).

The sum $a + b + c$ can evidently be derived by adding up all shares.

It does not matter (in this case) how shares are distributed.

After sharing out, each party can compute $s_i = a_i + b_i + c_i$, and the final sum is obtained by reconstruct on s_1, s_2, s_3.



MPC IN GENERAL VS MPC ON A CHIP

In the general MPC setting, parties are often “separate entities” (although perhaps their computations end up being co-located on some cloud server).

These separate entities might be “taken over” by adversaries, in which case an adversary sees everything that this party is doing (and depending on the precise definition, they may also interfere with a party is doing, but we get to this later).

In the context of embedded system security we are not suggesting that we now implement cryptography across multiple devices.

Instead, we are, somewhat curiously, “equating” intermediate variables (i.e. things that occur during the computation of say AES) as “secret values” and we seek to share these out on the same device.

ROADMAP

MPC
- Definitions
- Application as Side Channel Countermeasure

Secret Sharing for Probing Security
- Addition
- Special Multiplications
- General Multiplication
- ISW: “probing-secure” multiplication for (d,d) Boolean secret sharing

Parties are Variables/Wires

In the side channel scenario, the adversary can observe timing, power, EM, etc.

These side channels can be understood as functions that gives some information about (all) intermediate steps that occur during some computation.

A step, is a variable, say x, e.g. the result of the Sbox lookup on a state byte, $x = \mathrm{S}(s_j \oplus rk_j)$.

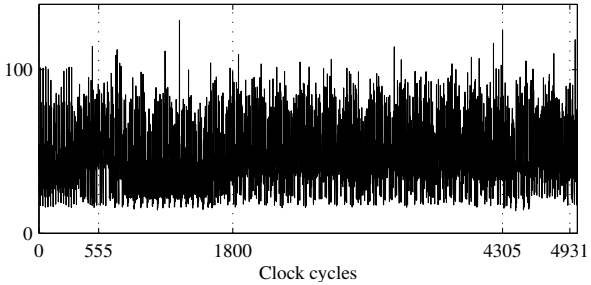
We now make the mental leap that rather than working on variables like x, s_j, rk_j we want to work on variables that represent their shares.

So, the parties that we consider in the context of side-channel countermeasures, are variables that represent shares. They are all "kept" on one device.

Leakage of an Intermediate Variable

In differential input attacks, we predict the value of some key-dependent intermediate variable (i.e. the x in the previous slide), and compare the predictions with the actual behaviour of the device (by using some side channel).

We compare the predictions for each key candidate k_j with the $i-th$ trace point (for all trace points), via estimating the correlation $\hat{\rho}_{k_j}(\mathbf{H}_j, \mathbf{T}_i) = \hat{\rho}_{k_j}(M(\mathbf{V})_j, \mathbf{T}_i)$.



Leakage of an Intermediate Variable, cont.

DPA reveals the key via: $\hat{\rho}_{k_j}(\mathbf{H}_j, \mathbf{T}_i) = \hat{\rho}_{k_j}(M(\mathbf{V}_j), \mathbf{T}_i)$.

H_i is based on a "power model" M to map the predicted values of the intermediate variable to some "hypothetical power model".

The actual device leakage is "noisy" (i.e. many circuit parts contribute which are independent of the computation that we are interested in): $T_i = \mathcal{L}(V(s,k)) + \mathcal{N}$. $M \neq \mathcal{L}$ in practice, so the adversary that has an $M = \mathcal{L}$ is extremely lucky/good, and an adversary that can retrieve the actual intermediate values is the strongest that we can imagine.

The "strongest" adversary would have access to "raw intermediate values" (taken by some specific intermediate variables). We say that they can "probe" them.

Leakage on Shares

Suppose that an intermediate bit variable x leaks its value: if $\mathcal{A}$ observes the leakage of x, then they know exactly what values x has taken.

Now suppose that we Boolean secret share the bit x into two shares: $x = (x_1, x_2)$. Obviously if we give the adversary leakage on both shares, then they can work out x.

But what happens if we restrict them to observing just one variable? Well, we know that the distribution of each share x_i is independent of x, so they learn nothing about x.

An adversary that can select to see either x_1 or x_2 learns nothing about x: for all outcomes of x_i, the probability that $x = 0$ is the same as the probability that $x = 1$.

x	x_1	x_2
0	0	0
0	1	1
1	0	1
1	1	0

Probing security of $(2,2)$-Boolean SS scheme for a Function $F(a,b)$ (Informal Idea)

Informally: An adversary may select one **single variable** v_j occurring during the $(2,2)$-shared computation of a function $F(a,b)$: yet, they cannot infer any information about a or b.

Link with key recovery: in the context of an encryption function, a, b are elements of the state, which are key dependent.

If the adversary learns nothing about a, b, then they also learn nothing about the key.

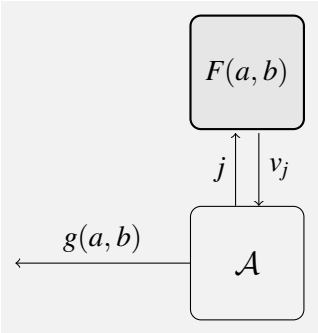


Figure: Probing security for $d = 2$.

Over to You

What does an adversary learn with different combination functions to create "joint distributions" of x_1 and x_2?

x	x_1	x_2	$x_1 + x_2$	$x_1 \cdot x_2$	$(x_1 - x_2)^2$
0	0	0			
0	1	1			
1	0	1			
1	1	0			

Suppose that a trace has 1000 points. The adversary knows that two of these points are the leakage of x_1 and x_2. What is their effort to exploit the leakage (using perhaps on of the combination functions in the table)?

Boolean Secret Sharing as a Countermeasure

The idea is to represent the message m and the key k via d shares, and ensure that computations on shares are done independently of each other (not easy to ensure in practice).

Then, by the properties of Boolean secret sharing we know that an adversary who can observe no more than $d - 1$ shares, will learn nothing about m, k.

What if an adversary gets to observe all d shares?

- In practice this is possible by selecting and jointly processing d trace points
  - if the adversary does not know these d points, they need to exhaustively try the $1/2 \cdot \binom{t}{d}$ combinations of points;
  - the joint distributions of d trace points is often derived via multiplying them: the resulting points have a variance that is exponentially higher than the raw trace points.
- The adversary's effort potentially increases by an exponential factor d (for d large enough, attacks become computationally infeasible).

Deep learning can be more effective in dealing with implementations on shares.

PROBING SECURITY FOR ANY d

An adversary that may select $d-1$ intermediate variables $\mathcal{V}$ that occur during the computation of a function $F(a,b)$, cannot learn any information about a or b.

We express the "cannot learn any information" via the RoR game.

In the context of an encryption function, a, b are elements of the state, which are key dependent.

If the adversary learns nothing about a, b, then they also learn nothing about the key.

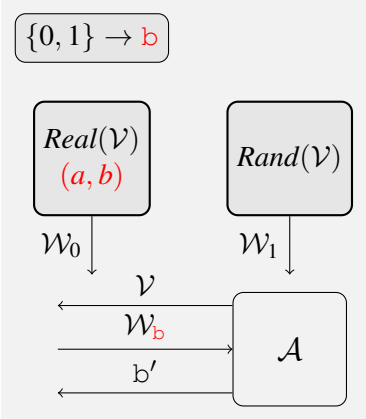


Figure: Probing security for d; $\mathcal{A}$ wins if $\mathtt{b} = \mathtt{b}'$

Roadmap

MPC
- Definitions
- Application as Side Channel Countermeasure

Secret Sharing for Probing Security
- Addition
- Special Multiplications
- General Multiplication
- ISW: “probing-secure” multiplication for (d,d) Boolean secret sharing

Block cipher arithmetic

We wish to split the message and the key of an encryption function like AES up into d shares.

Hence we need to represent the round function and key schedule as some Boolean operations.

We have done this for AES already: we know that AddRoundKey can be understood as Boolean addition, and the complicated functions SubBytes and MixColumns are in fact functions over an extension field of $\mathbb{Z}_2$, so can also be done as Boolean operations, but here we need the multiplication as well. ShiftRows only requires to rearrange the state.

Other encryption functions can equally be represented as a combination of additions and multiplications over some $\mathbb{Z}_{2^n}$.

MPC Addition in $\mathbb{Z}_{2^n}$

We have wish to compute $c = a \oplus b$, with $a, b, c \in \mathbb{Z}_{2^n}$: we split a and b into d shares; then we add the respective shares, eventually we reconstruct.

```
1  def add(a,b):
2    c = a ⊕ b;
3  return c
```

Listing: $a + b$

```
1  (a_1, a_2, ..., a_d) = Bdd.Share(a, d);
2  (b_1, b_2, ..., b_d) = Bdd.Share(b, d);
3
4  def add((a_1, a_2, ... a_d), (b_1, b_2, ..., b_d))
5    for i = 1 to d do:
6      c_i = a_i ⊕ b_i
7    return (c_1, c_2, ..., c_d)
8
9  c = Bdd.Reconstruct(c_1, c_2, ..., c_d)
```

Listing: $a + b$

Notice that the shares of a and the shares of b are independent.

Probing Security Argument for MPC Addition

Proving security means to prove that we can write a simulator for the Random world that an adversary cannot distinguish from the Real world.

We want to argue that no matter which $d-1$ intermediate variables an adversary wants to probe (i.e. see the values of), we can produce them with our simulator.

So we run through all "cases" and argue that we can always give an answer that "looks correct", and has the same distribution as what the Real world would produce.

```
1  (a_1, a_2, ..., a_d) = Bdd.Share(a, d);
2  (b_1, b_2, ..., b_d) = Bdd.Share(b, d);
3
4  def add((a_1, a_2, ... a_d), (b_1, b_2, ..., b_d))
5    for i = 1 to d do:
6      c_i = a_i ⊕ b_i
7    return (c_1, c_2, ..., c_d)
8
9  c = Bdd.Reconstruct(c_1, c_2, ..., c_d)
```

Listing: $a + b$

PROBING SECURITY ARGUMENT FOR MPC ADDITION

Assume that the adversary can ask for any $d-1$ shares occurring during the addition of shares:

```
1 def add((a_1, a_2, ... a_d), (b_1, b_2, ..., b_d))
2   for i = 1 to d do:
3     c_i = a_i ⊕ b_i
4   return (c_1, c_2, ..., c_d)
```

Listing: $a+b$

Case: only shares from a: due to the properties of Boolean secret sharing that with $d-1$ shares of a the adversary learns nothing about a, all share are independently and uniformly distributed.

Case: only shares from b: same argument as before.

Case: only shares from c: same argument as before (if c_d is included then we know that this is akin to adding two independent one-time encryptions, which preserves the property

PROBING SECURITY ARGUMENT FOR MPC ADDITION, CONT.

Assume that the adversary can ask for any $d-1$ shares occurring during the addition of shares:

```
1  def add((a_1, a_2, ... a_d), (b_1, b_2, ..., b_d))
2    for i = 1 to d do:
3      c_i = a_i ⊕ b_i
4    return (c_1, c_2, ..., c_d)
```

Listing: $a+b$

Case: shares from a and b: same argument as before (because we required that they are independent).

Case: shares from a and c: if they see a_i, c_i then the adversary can deduce the corresponding b_i, but this gives them no information about other b_j, so they still learn nothing.

Case: shares from b and c: as in the previous case.

Probing security argument for MPC addition, cont.

Assume that the adversary can ask for any $d-1$ shares occurring during the addition of shares:

```
1 def add((a_1, a_2, ... a_d), (b_1, b_2, ..., b_d))
2   for i = 1 to d do:
3     c_i = a_i ⊕ b_i
4   return (c_1, c_2, ..., c_d)
```

Listing: $a+b$

Case: shares from a, b, and c: if they ask for the i-th share from all three, then they get redundant information, so no additional information; if they ask for different shares from the three, then the arguments from before apply once more.

Over to You

Convince yourself that $c_d = a_d \oplus b_d$ is uniformly distributed.

Probing security: Simulator for Random World

```
1  Real-oracle(V):
2  if |V| > d − 1:
3    return ⊥
4  for i = 1 to d − 1:
5    a_i ←{0,1}^l
6    b_i ←{0,1}^l
7  a_d = a ⊕ ∑_{i=1}^{d−1} a_i
8  b_d = b ⊕ ∑_{i=1}^{d−1} b_i
9  for i = 1 to d do:
10   c_i = a_i ⊕ b_i
11 return {a_i, b_i, c_i|V}
12
```

```
1  Random-oracle(V):
2  if |V| > d − 1:
3    return ⊥
4  for i = 1 to d:
5    if a_i ∈ V: a_i ←{0,1}^l
6    if b_i ∈ V: b_i ←{0,1}^l
7  for i = 1 to d do:
8    if (a_i, b_i, c_i) ∈ V:
9      c_i = a_i ⊕ b_i
10   else
11     c_i ←{0,1}^l
12 return {a_i, b_i, c_i|V}
```

The Random world simply samples all shares as random values, and ensures "consistency": if the adversary asks for a_i, b_i, c_i (with the same i), then the c_i is indeed the sum of the corresponding a_i, b_i. We have argued on the slides before, why, all of the cases result in independent and uniform shares.

Over to You

Consider how you would compute $3 \cdot a$ securely.

Roadmap

MPC
- Definitions
- Application as Side Channel Countermeasure

Secret Sharing for Probing Security
- Addition
- Special Multiplications
- General Multiplication
- ISW: “probing-secure” multiplication for (d,d) Boolean secret sharing

Repeated Addition of x: $a \cdot x$

The addition of two values is essentially a component-wise operation. The corresponding input shares are added (modulo two): $(a_1, a_2, \dots) + (b_1, b_2, \dots) = (a_1 \oplus b_1, a_2 \oplus b_2, \dots)$. (Bear in mind that each a_i, b_i is a polynomial with coefficients in $\mathbb{Z}_2$.)

$$a \cdot x = a \cdot (x_1 \oplus \cdots \oplus x_d) = \underbrace{(x_1 \oplus \cdots \oplus x_2) \oplus \cdots \oplus (x_1 \oplus \cdots \oplus x_2)}_{a \text{ times}} = (ax_1 \oplus \cdots \oplus ax_d)$$

Hence we can carry out the multiplication of a Boolean string (Finite Field element) x with a constant as:

```
1  Bdd.MultConst(a, (x_1, x_2, ..., x_d)):
2    for i = 1 to d:
3      c_i = a · x_i
4    return (c_1, c_2, ..., c_d)
```

Notice that we do not need to carry out a polynomial reduction here because the repeated addition of an element with itself cannot change the degree of the polynomial.

SQUARE OF x: $x \cdot x$

Recall: $x_i = a_0 \oplus a_1 x \oplus a_2 x^2 \ldots$, whereby the a_i are in $\mathbb{Z}_2$, and all operations on polynomials x_i are done over some extension $\mathbb{Z}_{2^n}$ (so via some irreducible polynomial $p(x)$).

Notice that $(a \oplus b)^2 \equiv a^2 \oplus b^2$ in $\mathbb{Z}_2$ because all other terms in the expansion of $(a \oplus b)^2$ are multiples of 2 and hence they are equivalent to 0 in $\mathbb{Z}_2$. This implies that $x = (a_0 \oplus a_1 x \oplus \ldots)^2 \equiv a_0^2 \oplus a_1^2 x^2 \oplus \ldots$ in $\mathbb{Z}_{2^n}$.

The square of a polynomial over $\mathbb{Z}_{2^n}$ is therefore also a component-wise operation: $x^2 = (x_1 \oplus \cdots \oplus x_d)^2 = (x_1^2 \oplus \cdots \oplus x_d^2) \pmod{p(x)}$

```
1  Bdd.Square((x_1, x_2, ..., x_d)):
2    for i = 1 to d:
3      c_i = (x_i)^2 (mod p(x))
4    return (c_1, c_2, ..., c_d)
```

ROADMAP

MPC
- Definitions
- Application as Side Channel Countermeasure

Secret Sharing for Probing Security
- Addition
- Special Multiplications
- General Multiplication
- ISW: “probing-secure” multiplication for (d,d) Boolean secret sharing

MULTIPLICATION FOR $d = 2$

Let's try and develop an "intuition" for multiplication.

We want to have $c = a \cdot b$ (in some $\mathbb{Z}_{2^n}$). So the multiplications are all finite field multiplications.

But we have to work with shares so $(a_1 \oplus a_2) \cdot (b_1 \oplus b_2)$ (and we must not actually compute the additions here, otherwise we leak a and b).

We write the shares of a along the rows, and the shares of b along the columns of a $2x2$ matrix, and the products $a_i b_j$ in the respective entries.

	b_1	b_2
a_1	$a_1 b_1$	$a_1 b_2$
a_2	$a_2 b_1$	$a_2 b_2$

MULTIPLICATION FOR $d = 2$, CONT.

Let's try and develop an "intuition" for multiplication.

How do we sum up the partial products to create the shares c_1, c_2 of the product?

Let's sum across the columns:

	b_1	b_2	
a_1	a_1b_1	a_1b_2	$c_1 = a_1b_1 \oplus a_1b_2$
a_2	a_2b_1	a_2b_2	$c_2 = a_2b_1 \oplus a_2b_2$

Can you spot the problem?
c_1 can also be written as $a_1 \cdot (b_1 \oplus b_2) = a_1 \cdot b$, and $c_2 = a_2 \cdot b$.
Therefore if $b = 0$, the no matter what values a_i take, c_i will equal zero.
Hence, $\Pr[c_1, c_2 = 0 | b = 0] = 1$ and if an adversary plays this game multiple times with some fixed a, b, then they will eventually learn if $b = 0$ or not.

This may not be much of an information leak, but it violates the security notion.

MULTIPLICATION FOR $d = 2$, CONT.

How about when we diagonalise the matrix, so we "mix up" the columns:

	b_1	b_2	
a_1	a_1b_1	a_1b_2	$c_1 = a_1b_1 \oplus a_1b_2$
a_2	a_2b_1	a_2b_2	$c_2 = a_2b_1 \oplus a_2b_2$

"Flip" the element above the diagonal to the "opposite" one below:

	b_1	b_2	
a_1	a_1b_1		$c_1 = a_1b_1$
a_2	$a_2b_1 \oplus a_1b_2$	a_2b_2	$c_2 = a_2b_2 \oplus a_2b_1 \oplus a_1b_2$

The c_i do not appear to so strongly depend on either a or b any more.

But is it actually uniform? And how do we deal with having to compute an addition with three terms?

Over to You

Consider how to compute $c_2 = a_2b_2 \oplus a_2b_1 \oplus a_1b_2$ using Boolean addition (two inputs). How many different ways are there to do this computation? Are all equally secure/insecure?

Over to You

I claim that that the distribution of c_2 is not uniform across a, b. Below is an illustration for the case where a, b are bit values, and $d = 2$. Check if you can complete the missing parts of the table. What is the reason for the non-uniform distribution of c_2?

a	b	$\Pr[c_2 = 0]$	$\Pr[c_2 = 1]$
0	0	$3/4$	$1/4$
0	1	$3/4$	$1/4$
1	0		
1	1	$1/4$	$3/4$

MULTIPLICATION FOR $d = 2$, CONT.

To deal with the problem we need to introduce additional, independent randomness, which we call $r_{1,2}$.

We place it into the empty space and "flip" it as well so that we get a correct result.

	b_1	b_2	
a_1	$a_1 b_1$	$r_{1,2}$	$c_1 = b_1 a_1 \oplus r_{1,2}$
a_2	$(r_{1,2} \oplus a_1 b_2) \oplus a_2 b_1$	$a_2 b_2$	$c_2 = a_2 b_2 \oplus ((r_{1,2} \oplus a_1 b_2) \oplus a_2 b_1)$

Notice that I have "grouped" certain computations by placing brackets around them.

This indicates that there is an order to the computations, for good reason, as you worked out before in one of the exercises.

MULTIPLICATION FOR $d = 3$

The starting point here would be the following partial products:

	b_1	b_2	b_3	
a_1	a_1b_1	a_1b_2	a_1b_3	c_1
a_2	a_2b_1	a_2b_2	a_2b_b	c_2
a_3	a_3b_1	a_3b_2	a_3b_3	c_3

I colour coded the elements above the diagonal in green. The result of the multiplication is collected via summing across columns, leading to the same issue around the non-uniformity of the partial products.

MULTIPLICATION FOR $d = 3$, CONT.

Like for $d = 2$ the sum over the partial products across the columns is not uniformly distributed, so we need to introduce fresh randomness $r_{i,j}$.

	b_1	b_2	b_3	
a_1	a_1b_1	$r_{1,2}$	$r_{1,3}$	c_1
a_2	$(r_{1,2} \oplus a_1b_2) \oplus a_2b_1$	a_2b_2	$r_{2,3}$	c_2
a_3	$(r_{1,3} \oplus a_1b_3) \oplus a_3b_1$	$(r_{2,3} \oplus a_2b_3) \oplus a_3b_2$	a_3b_3	c_3

In the next slide, we'll see a general algorithm that performs this process for any d. It samples the randomness $r_{i,j}$, and then computes the terms in the brackets and calls them $r_{j,i}$.

	b_1	b_2	b_3	
a_1	a_1b_1	$r_{1,2}$	$r_{1,3}$	c_1
a_2	$r_{2,1} \oplus a_2b_1$	a_2b_2	$r_{2,3}$	c_2
a_3	$r_{3,1} \oplus a_3b_1$	$r_{3,2} \oplus a_3b_2$	a_3b_3	c_3

Roadmap

MPC
- Definitions
- Application as Side Channel Countermeasure

Secret Sharing for Probing Security
- Addition
- Special Multiplications
- General Multiplication
- ISW: “probing-secure” multiplication for (d,d) Boolean secret sharing

Ishai, Sahai, Wagner (ISW) Multiplication Algorithm

The randomness used for a must be independent of the randomness used for b. Both inputs are l-bit values. Multiplications are over some finite field.

```
def SecMult({a_1, a_2, ..., a_d}, {b_1, b_2, ..., b_d}):
    for i = 1 to d do:
        for j = i + 1 to d do:
            r_{i,j} ← {0,1}^l
            r_{j,i} = (r_{i,j} ⊕ a_i b_j) ⊕ a_j b_i
    for i = 1 to d do:
        c_i = a_i b_i
        for j = 1 to d do:
            if j ≠ i:
                c_i = c_i ⊕ r_{i,j}
    return {c_1, c_2, ..., c_d}
```

ISW SECURITY CLAIM

Given independent sharings of l-bit values a and b, any $(d-1)$ tuples of intermediate values in the SecMult algorithm, can be perfectly simulated by random values.

This means that we can build a simulator that represents the Random world, and that this simulator will produce outputs that no adversary can distinguish from what would be produced in the Real world.

We will show this fact by demonstrating that any $(d-1)$ tuple of intermediate variables in SecMult (including input and output shares) only depends on (at most) $d-1$ shares a and $d-1$ shares of b. Because $d-1$ (out of d) shares are independent of a (and b), we can sample them uniformly, thus do not need to know a, b.

ISW PROOF (SKETCH)

We play the same probing game (RoR) as before.

The adversary's job is to distinguish correctly between Real and Random world outputs.

The adversary can now select up to $d - 1$ intermediate variables by supplying a list, which I call $\mathcal{V}$ in this picture.

They then get a response from either the Real or the Random world and have to decide which world produced the response.

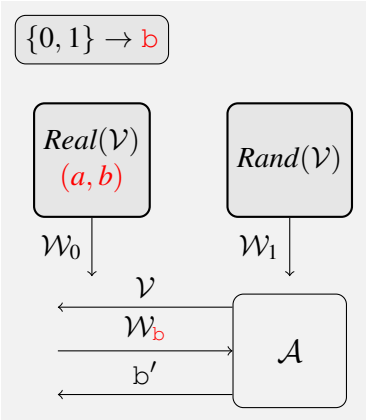


Figure: Probing security for d; $\mathcal{A}$ wins if $\mathtt{b} = \mathtt{b}'$

ISW PROOF (SKETCH)

In the Real world, SecMult is computed with some input a, b. So every time SecMult is run, new shares for a, b are produced.

The Random world does not have knowledge of a, b. Therefore, we must show that we can simulate any combination of $d - 1$ intermediate variables of SecMult, using only random values.

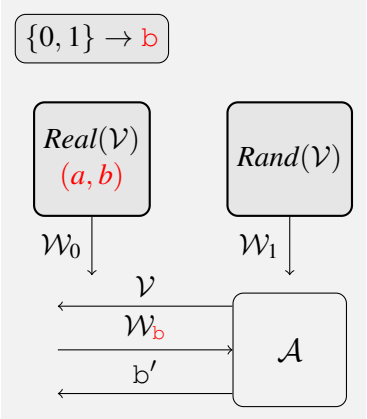


Figure: Probing security for d; $\mathcal{A}$ wins if $\mathtt{b} = \mathtt{b}'$

EXAMPLES

The following intermediate variables are produced by SecMult:

$a_i, b_i, r_{i,j}, r_{i,j} \oplus a_i b_j, r_{j,i} = (r_{i,j} \oplus a_i b_j) \oplus a_j b_i$

$a_i b_i, a_i b_i \oplus r_{i,j}$ and further sums of $a_i b_i$ and $r_{i,j}$.

From now on we call the tuple, that the adversary selects, $\mathbf{v}_A$. We look at some examples of $\mathbf{v}_A$, if SecMult rans with $d = 4$.

$\mathbf{v}_A = (a_2 b_2, r_{3,2}, r_{2,4})$:
We sample random values for a_2 and b_2, and produce $a_2 b_2$. The variable $r_{3,2}$ depends on $r_{2,3}$ and a_2 and b_3, but $r_{2,3}$ has not been requested, so we replace it with a random value. The variable $r_{2,4}$ is an independent random variable in SecMult, and there are no dependencies with other variables that have been requested so we sample it as a random variable.

EXAMPLES, CONT.

$\mathbf{v}_A = (r_{4,1}, r_{4,2}, r_{2,4})$:
The value $r_{4,1} = r_{1,4} \oplus a_1 b_4$; the value $r_{4,2} = r_{2,4} \oplus a_2 b_4$. For $r_{4,1}$ we observe that none of the variables that it depends on are in the list so we can return a random value. The other two values have a dependency, but $a_2 b_4$ is not in the list. So we could sample both as random, or, sample the "right side" and produce $r_{4,2}$.

$\mathbf{v}_A = (a_1 b_2, r_{2,1}, a_1 b_1)$:
Here we have multiple dependencies: we samples random values a_1, b_1, b_2 so we can produce $a_1 b_2$ and $a_1 b1$. The variable $r_{2,1} = r_{1,2} \oplus a_1 b_2$ can be produced by using what we already sampled plus a random value $r_{1,2}$.

Simulation Strategy

We could try and write a program that exhaustively lists/runs through all "cases": some tools verify probing proofs in such a way.

However, in their landmark paper, ISW argue in a neat way of how to deal with any request, based on the following observations:

- Intermediate steps like sums and products have two input variables. (Any line with multiple operators must be understood as multiple steps).
- Whilst $r_{i,j}$ and $r_{j,i}$ are, individually taken distributed uniformly, and independently of a_i, a_j, b_i, b_j their joint distribution with $a_i b_j$ is not independent, and we must produce them according to the algorithm.
- We can keep track of the shares that may be required to compute a valid response, by maintaining two lists: $\mathcal{I}$ for the shares of a, $\mathcal{J}$ for the shares of b.

ISW PROOF (SKETCH), BOOK KEEPING

First step: for each element v in $\mathbf{v}_A$ we are going to make the following assignment:

- If v is one of the variables $a_i, b_i, a_i b_i, r_{i,j}$, or if it is a sum of them (this includes the c_i), then, add the index i of it to $\mathcal{I}$ and also to $\mathcal{J}$.
- If v is one of the $a_i b_j$ or $r_{i,j} \oplus a_i b_j$, then we add i to $\mathcal{I}$ and j to $\mathcal{J}$.

Examples for $d = 4$.

- $\mathbf{v}_A = (a_2 b_2, r_{2,3} \oplus a_2 b_3, a_2)$: then the sets are $\mathcal{I} = \{2, 3\}$ and $\mathcal{J} = \{2, 3\}$
- $\mathbf{v}_A = (a_1, a_2, r_{3,4} \oplus a_3 b_4)$: then the sets are $\mathcal{I} = \{1, 2, 3\}$ and $\mathcal{J} = \{1, 2, 4\}$
- $\mathbf{v}_A = (a_3, r_{3,4}, r_{3,4} \oplus a_3 b_4)$: then the sets are $\mathcal{I} = \{3\}$ and $\mathcal{J} = \{3, 4\}$
- $\mathbf{v}_A = (r_{1,4} \oplus a_1 b_4, r_{2,4} \oplus a_2 b_4, r_{2,4})$: then the sets are $\mathcal{I} = \{1, 2, 4\}$ and $\mathcal{J} = \{1, 2, 4\}$

Notice that in all cases both lists contain at most $d - 1$ indices. Hence the shares that corresponds to these indices can be derived by sampling from a uniform distribution, because $d - 1$ out of d shares are independent of the unshared value.

ISW PROOF (SKETCH), CONT.

Second step: tackle the first part in SecMult, which generates the $r_{i,j}$. This runs through the indices i, j, so we can have four cases:

- $i \notin \mathcal{I}$: implies $r_{i,j}$ is not in $\mathbf{v}_A$, so we don't care.
- $i \in \mathcal{I}, j \notin \mathcal{J}$: so $r_{i,j}$ might be required (if $i < j$), but as $j \notin \mathcal{J}$ we know that $r_{j,i}$ is not in $\mathbf{v}_A$, hence we assign a random value ot $r_{i,j}$.
- $\{i,j\} \in \mathcal{I}$ but $\{i,j\} \notin \mathcal{J}$: this means that only $r_{i,j}$ is on the list, and hence it can be sampled as a random value.
- $\{i,j\} \in \mathcal{I}$ and $\{i,j\} \in \mathcal{J}$: both $r_{i,j}$ and $r_{j,i}$ have been requested, so we produce them according to the algorithm.

ISW PROOF (SKETCH), CONT.

Third step: we show how to simulate all (other) intermediate variables.

- Consider the intermediate variables of the form $a_i, b_i, a_i b_i, r_{i,j}$ (including $j > i$), and sums of them including the c_i as a special case. If the adversary requested them then we have $i \in \mathcal{I}$ and $\mathcal{J}$ and we already assigned values to $r_{i,j}$ in the previous step. So any intermediate variable in this list can be perfectly simulated.
- Consider the intermediate variable of the form $a_i b_j$ and $r_{i,j} \oplus a_i b_j$. If the adversary requested them, then we have $i \in \mathcal{I}$ and $j \in \mathcal{J}$. So we can simulate $a_i b_j$ and we did already assign values to $r_{i,j}$ in the previous step. Therefore we can perfectly simulate these intermediate variables as well.